Haya: Live-Compute-Index-Guided, Swap-Cost-Aware Routing for On-Device LLMs

Karthikeyan N
Assistant Professor
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
karthikeyan.n@rajalakshmi.edu.in

Monic Auditya A
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701194@rajalakshmi.edu.in

Atchaya S
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701045@rajalakshmi.edu.in

Saranya M
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701510@rajalakshmi.edu.in

Ramya Sreevarshini B
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Chennai, India
230701262@rajalakshmi.edu.in

***Abstract*—The primary trade-off involves the level of task-device state alignment and the associated overhead for resident model update. This paper describes Haya, an offline solution that: (i) calculates the Live Compute Index (LCI) for each candidate; (ii) performs feasibility tests with respect to the available memory; and (iii) finds out the amortized overhead required to switch models and reconstruct history for each candidate, making a switch possible only if the gains achieved in both cases exceed the hysteresis threshold. Three Q4_0 models have been considered by using a replay dataset containing 200 prompts on Realme GT 6T and Realme 11 Pro+ devices. Compared to Complexity-Only, Haya decreases latency and energy consumption by 29.6% and 17.2%, respectively, on GT 6T, and 32.1% and 20.7%, respectively, on Realme 11 Pro+, decreasing the number of switches from 41 to 11, and from 44 to 13, respectively. Human-judgment quality in terms of means is 0.746 as compared to 0.724 on GT 6T, and 0.718 as compared to 0.724 on Realme 11 Pro+. An experiment conducted on GT 6T shows that without considering the swap-cost factor, the number of model switches increases from 11 to 38, and the average latency increases from 3,253 ms to 3,820 ms. Timings show that reconstruction of 1,024 history tokens requires 85% of the total time 29.12 seconds allocated for handoff.**

***Index Terms*—On-device inference, model routing, model switching, swap cost, smartphone language models.**

I. INTRODUCTION

Performing inference locally guarantees that prompts will be on-device; however, smartphones have strict limitations concerning memory, energy, and thermal dissipation [1]–[4]. The models used may depend not only on the type of the current query but also on the state of the device. The score for the query does not include the overhead costs of the switching process, which involves unloading of the previous model, loading of the new one, setting up the execution environment, and, importantly, re-encoding of the dialogue history. Existing research on mobile large language models and multi-turn routing algorithms stresses the importance of memory pressure, model switching, and overheads related to context rebuilding during the transition between models [1], [5]–[7]. The overheads related to the handoff process may exceed the possible gains for the current query, especially in cases of short sessions or lengthy dialogues.

While existing routers decide about the proper model for a query based on such parameters as query complexity, thermal state, or hardware load, they assume that the chosen model is available at zero cost, while mobile LLMs are faced with the memory consumption of one resident model. To the best of our knowledge, there is no prior art that combines multiple models on one device with live temperature, battery, and memory states, model switching overheads, and offline heuristic for model switching. Our literature survey revealed no precedents in this matter.

Haya scores the candidate models according to the Live Compute Index (LCI) and how much the models match the particular query, removes models failing the memory reservation test, and amortizes the costs of switching and context rebuilding over the number of turns. The main replay runs models one by one.

Contributions

This paper makes the following four important contributions. First, we frame the decision of whether to stick to locally cached models or switch to other models in terms of the local cache index (LCI) of individual models, their fit to the query, fit to memory, and amortized switching cost, under memory admission constraints and hysteresis margin. Second, we estimate the switching cost between two models on two smartphones in terms of flash loading cost, initialization cost, and history re-prefill cost, and calculate break-even turns for switching from those costs. Third, we compare our switching policy to fixed model, Complexity-Only, Thermal-Aware, reactive LCI, and offline oracle switching policies through a 200-prompt replay test. Fourth, we run several ablations, swapping cost sensitivity analysis, sustained thermal tests, proactive switching, and memory residency tests to identify the factors that drive our performance gain.

The rest of the paper is organized as follows. Section II summarizes related work. Section III introduces our policy. Section IV describes our experimental methodology. Section V shows our results. Section VI discusses the limitations and future work. Section VII draws conclusions.

II. LITERATURE SURVEY

Inference Optimization Techniques for Mobile Devices optimize for optimization, quantization, flash-backed weights, and memory management for models residing on the device [1]–[4], [8]–[16]; however, they resolve the issue of managing just one resident model but not the selection among many resident models. Difficulty-Driven Routers divert easy queries

to poor performing models and difficult queries to good performing models [17]–[19]; Adaptive Orchestration and Resource-Aware Routing take into account the live status of the system [20]–[25]. HybridInfer [22] takes into consideration the thermal headroom as well as the difficulty of requests in selecting on-device, edge, or cloud tier for inference; however, However, it is constrained to maintaining one model at the phone level without any additional cost. The most related works for GPU service problems include HW-Router [23] and Measured Joules, Learned Routes [24], but the work of EnerInfer [25] is more concerned about temperature and frequency management rather than downloading model selection. In relation to history of the conversation and the possibility of using different models due to availability, Dynamic Quality-Latency Aware Routing incorporates the costs of switching and KV-cache costs in the mobile-edge route, but it has only one small model at the phone level [5]; MTRouter considers model utility in regard to conversation history [6]; cross-model KV cache transfer reduces the cost of history reading for only one family of models on server-class models [7]; MemSpec maintains resident draft models for speculative decoding [26]; and mzCache maintains model state for mobile multitasking [16].

Examples of studied problems in literature include routing, thermal management, memory allocation, and context switching, but this paper does not make any general claim about the performance of the first router. In contrast, Haya addresses a much more narrow question: what role do flash loading and history building costs play in decision making for a system where only one model is loaded per query?

III. PROPOSED MODEL

Haya is a per-query keep-or-change approach for an offline phone where multiple models exist, each of different sizes. Haya determines the query's hardness, examines the machine, ranks all candidates and decides whether to serve the query using the current resident model or pay to switch to another (see Figs. 1 & 2).

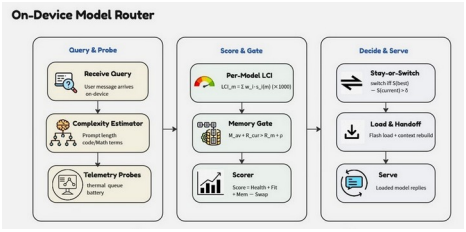


Fig. 1. Haya's on-device routing stages: query and telemetry probes, per-model scoring and memory admission, and the stay-or-switch decision before serving.

A. QUERY AND DEVICE PROBES

For each request, a lightweight classifier is used to compute a complexity value c ∈ [0, 1], based on surface features such as prompt length and use of programming/math keywords; the probe period reports thermal headroom, battery headroom, memory headroom, and local queue information; the thermal headroom calculation is based on the Android Power Manager API [27].

A request uses the latest sample rather than waiting for a new probe. Each model's LCI aggregates normalized device and model signals:

(1)

where si(m) ∈ [0, 1] is the normalized values for decode speed, memory margin, temperature, battery, and local queue length. Normalization algorithm and the weights associated with each signal are implementation specific and will be covered in Section VI.

B. FROM LCI V1 TO HAYA

Haya improves upon an initial proposal based on ranking memory feasible candidates based on S = 0.55 LCI + 0.33 ModelQuality + 0.12 MemoryFit, where LCI is only determined for the phone. There are some deficiencies with this formulation for multi-model routing: the LCI value at a phone level provides the same benefit to all candidates and does not affect their order; the query is never analyzed; there is nothing to indicate which model is loaded on the phone, thus allowing for switchbacks; the network round-trip time works better for a remote server and not an offline phone; the values do not provide any information about how the weights were selected; the values are not normalized in a way to provide comparable scales; and the quality score does not consider that the quality is task-dependent.

C. SCORE, MEMORY GATE, AND SWITCH RULE

The memory gate admits a candidate only when unloading the current model can leave the candidate and safety reserve ρ in memory:

(2)

Each feasible model is scored by

(3)

where ℓ_m = LCI$_m$/1000, M_{av} is currently free memory, R_{cur} is memory released by unloading the resident model, and R_m is the candidate's peak memory requirement. We define memory fit as normalized post-admission slack, $MF(m) = (M_{\text{av}} + R_{\text{cur}} - R_m - \rho)/(M_{\text{av}} + R_{\text{cur}})$. Query fit penalizes under-capacity more than excess capacity:

Here a_m is model capability. The resident model has zero swap cost; for another model, measured load and context-rebuild costs are amortized across expected remaining turns b_N:

(4)

We use weights $w_L = 0.30$, $w_Q = 0.40$, $w_M = 0.10$, and $w_S = 0.20$. Let $m^* = \arg\max_m S(m)$ over the feasible set. Haya switches only when $S(m^*) - S(m_{\text{cur}}) > \delta$; Otherwise, it uses the resident model for the query. Switching off the resident model, loading of the candidate model, creating context, and generation of text take place. In the case where there is no appropriate candidate model, memory is freed, and once again it uses the smallest set of models. Should this fail, then it reports the error. The first replay involves just one resident model.



Fig. 2. Stay-or-switch decision state machine, including the memory retry path and an error response when no model fits.

D. SWAP-TIME MODEL

Measured switching time includes flash loading, initialization, and context re-prefill:

(5)

where time is in seconds, H is the number of history tokens, and 41.2 tokens/s is the measured re-prefill rate. For a quality gain ΔQ and additional per-turn latency Δt, cumulative utility after k turns is $U_k = k\Delta Q - \lambda(T_{swap} + k\Delta t)$. The break-even turn is $k^* = \lambda T_{swap}/(\Delta Q - \lambda\Delta t)$ when the denominator is positive.

E. ILLUSTRATIVE EXAMPLE

Using fabricated values (no measurements): 3.2 GB is available after freeing up the Small resident model, ρ = 1.0 GB, δ = 0.05, and Large (4.4 GB) does not pass the gate. For a complex coding request (c = 0.7), Medium is worth 0.559 versus 0.393 for Small, and thus the router pays the approximately 12 s cost of reloading and switching. For chitchat (c = 0.2), Small is worth 0.655 versus 0.499, and thus it is retained. A device level LCI with a constant quality term would route both requests to the same model.

IV. IMPLEMENTATION AND EXPERIMENTAL SETUP

A. PLATFORM AND MODELS

The Realme GT 6T (Snapdragon 7+ Gen 3) and the Realme 11 Pro+ (Dimensity 7050) phones [28], [29] are employed in this research work. The assessed models include Qwen2.5-0.5B, Gemma-2-2B, and Phi-3-mini [30]–[32]; the tiers are four-bit Q4_0 GGUF implementations provided by the CPU implementation of llama.cpp with JNI [33], [34]. In total, ten models were assessed for each phone; please refer to Table I for further information. The measurement has been done independently 10 times per phone-model combination with temperature 33 °C in airplane mode starting with unthrottled burst mode condition. Gemma-2-2B has a total of 2.61B parameters [31].

TABLE I — Measured On-Device Inference Profiles for the Ten Candidate Models

Device and model	Params.	Load (s)	Decode (tok/s)	Peak RAM (MB)	Δ*T* (2 min, °C)	Battery (mAh/ 100 tok)
Realme GT 6T (Snapdragon 7+ Gen 3)						
SmolLM-360M	0.36B	0.85	47.6	339	+5.2	0.30
Qwen2.5-0.5B	0.5B	1.19	39.1	450	+7.0	0.43
TinyLlama-1.1B	1.1B	2.35	21.4	779	+9.4	0.95
Qwen2.5-1.5B	1.5B	3.22	16.3	1,044	+10.8	1.42
Gemma-2-2B	2.6B∗	4.15	12.4	1,398	+12.5	2.07
Phi-2	2.7B	5.24	9.6	1,842	+13.8	2.89
StableLM-3B	3.0B	5.80	8.7	2,013	+14.6	3.33
Qwen2.5-3B	3.0B	5.98	8.5	2,064	+14.8	3.44
Llama-3.2-3B	3.2B	6.31	8.1	2,142	+15.2	3.65
Phi-3-mini	3.8B	7.46	6.9	2,534	+16.5	4.55
Realme 11 Pro+ (Dimensity 7050)						
SmolLM-360M	0.36B	1.08	28.5	342	+6.2	0.52
Qwen2.5-0.5B	0.5B	1.48	24.3	455	+8.2	0.74
TinyLlama-1.1B	1.1B	3.02	14.8	788	+11.0	1.44
Qwen2.5-1.5B	1.5B	4.18	11.2	1,055	+12.6	2.16
Gemma-2-2B	2.6B∗	5.35	8.4	1,410	+14.6	3.20
Phi-2	2.7B	6.72	6.5	1,860	+16.1	4.46
StableLM-3B	3.0B	7.42	5.9	2,030	+16.9	5.11
Qwen2.5-3B	3.0B	7.65	5.8	2,085	+17.2	5.25
Llama-3.2-3B	3.2B	8.10	5.4	2,160	+17.6	5.72
Phi-3-mini	3.8B	9.45	4.8	2,560	+18.8	6.79

∗ Gemma-2-2B has 2.61B total parameters with tied input/output embeddings. Δ*T* is the measured chassis-temperature rise after two minutes of continuous inference.

B. WORKLOAD AND QUALITY JUDGING

During the evaluation step, 200 prompts are offered to the models in the following proportions: 45% of factual or conversational queries (Category A), 35% of structured reasoning (Category B), and 20% of mathematics or programming tasks (Category C). Five tests are conducted when the inter-prompt idle period is 4.0 s and the initial temperature is 33 °C. The response quality of LLMs is assessed by another LLM using MT-Bench-like [35] scoring criteria; the consistency of assessments with the reference one was estimated with Cohen's κ = 0.84 [36]. Neither of these information resources offers any data about the utilized prompt set and evaluation process. The numbers reported in Table I were calculated based on power rail readings for 100 generated tokens.

C. BASELINES AND RECORDED METRICS

Haya is compared to three baselines: Complexity-Only routing, Thermal-Aware routing similar to HybridInfer [22], LCI reactive policy (LCI v1), and an infeasible constraint oracle with off-line information of the sequence. We collect quality judgments, end-to-end latency with swaps, energy per query, maximum chassis temperature, number of switches, and swap latency.

V. RESULTS

A. ROUTING QUALITY AND DEVICE COST

Table II and Table III contain results of baselines in the fixed model setting, namely Complexity-Only, Thermal-Aware, reactive LCI, Haya, and constraint feasible mixed-integer oracle. Haya gets average judged quality of 0.746, average latency of 3,253 ms, 95th percentile latency of 6,180 ms, energy consumption of 28.8 J/query, maximum chassis temperature of 38.5 °C, and 11 changes of models for GT 6T device. When compared to Complexity-Only, average latency is decreased by 29.6%, energy is reduced by 17.2%, changes are decreased from 41 to 11, and swap time is decreased by 75%. The mean quality value is increased by 0.022, the standard errors are just descriptive, and no pair-wise significance claim is done. Haya is inferior to the offline oracle by 0.006 of quality and is slower by 333 milliseconds. Contrary to the Reactive LCI, Haya increases the mean quality by 0.098, reduces the latency by 15.7%, energy by 8.3%, change from 33 to 11, and peak temperature from 39.7 °C to 38.5 °C. Haya performs better than Fixed-Medium by 0.052 of quality, 51.8% latency, and 45.0% energy usage.

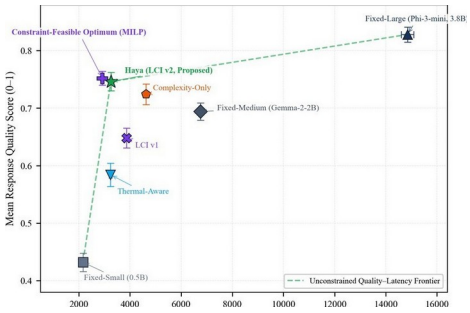


Fig. 3. Mean response quality against end-to-end latency for routing policies on the GT 6T. The constraint-feasible oracle uses non-causal sequence knowledge.

As far as the Realme 11 Pro+ smartphone is concerned, the corresponding Haya parameters are 0.718 in terms of quality, 4,920 ms for latency, 39.5 J/query for energy, and 13 changes of models as compared to 0.724 for quality, 7,250 ms for latency, 49.8 J/query for energy, and 44 changes of models in Complexity-Only. In such a case, Haya shows 32.1% lower latency, 20.7% lower energy consumption, and 70.5% less number of model changes while having slightly lower mean quality of 0.006 points. The maximum temperature of 39.8 °C does not exceed the permissible value of 40 °C, whereas it is exceeded in Complexity-Only and Fixed-Medium algorithms. The ratio of models in Haya mixing is 52/41/7% for GT 6T and 56/38/6% for Realme 11 Pro+.

TABLE II — Replay Results for 200 Mixed-Complexity Prompts:

Policy	Shares (%)	Quality	Mean (ms)	p_{95} (ms)
Realme GT 6T (Snapdragon 7+ Gen 3)				
Fixed-Small	100/0/0	0.432 ± 0.016	2,160 ± 42	2,840
Fixed-Medium	0/100/0	0.694 ± 0.015	6,750 ± 95	8,720
Fixed-Large	0/0/100	0.828 ± 0.013	14,850 ± 240	19,200
Complexity-Only	45/35/20	0.724 ± 0.018	4,620 ± 88	10,240
Thermal-Aware	74/24/2	0.584 ± 0.020	3,240 ± 62	5,680
LCI v1 (reactive)	52/38/10	0.648 ± 0.017	3,860 ± 75	7,390
Haya (proposed)	**52/41/7**	**0.746 ± 0.016**	**3,253 ± 54**	**6,180**
Constraint-feasible oracle	48/44/8	0.752 ± 0.012	2,920 ± 42	4,890
Realme 11 Pro+ (Dimensity 7050)				
Fixed-Small	100/0/0	0.432 ± 0.016	3,420 ± 58	4,450
Fixed-Medium	0/100/0	0.694 ± 0.015	10,200 ± 150	13,400
Fixed-Large	0/0/100	0.828 ± 0.013	21,400 ± 360	28,500
Complexity-Only	45/35/20	0.724 ± 0.018	7,250 ± 135	16,200
Thermal-Aware	78/20/2	0.565 ± 0.021	4,820 ± 88	8,450
LCI v1 (reactive)	56/34/10	0.635 ± 0.018	5,850 ± 115	11,800
Haya (proposed)	**56/38/6**	**0.718 ± 0.017**	**4,920 ± 82**	**9,400**
Constraint-feasible oracle	52/42/6	0.740 ± 0.013	4,450 ± 70	7,450

Shares are percentages in small/medium/large tier order. Values after ± are the reported standard errors; no uncertainty was supplied for p_{95}.

TABLE III — Replay Results for 200 Mixed-Complexity Prompts: Energy, Temperature, and Switching

Policy	Energy (J)	Peak (°C)	Switches	Swap (s)
Realme GT 6T (Snapdragon 7+ Gen 3)				
Fixed-Small	19.2 ± 0.5	37.2 ± 0.3	0	0.0
Fixed-Medium	52.4 ± 1.1	41.5 ± 0.4	0	0.0
Fixed-Large	124.5 ± 2.6	44.8 ± 0.5	0	0.0
Complexity-Only	34.8 ± 0.8	40.8 ± 0.4	41 ± 1.9	118.5 ± 5.2
Thermal-Aware	25.2 ± 0.6	38.2 ± 0.3	27 ± 1.6	63.2 ± 3.1
LCI v1 (reactive)	31.4 ± 0.7	39.7 ± 0.4	33 ± 1.8	85.2 ± 3.8
Haya (proposed)	**28.8 ± 0.6**	**38.5 ± 0.3**	**11 ± 1.1**	**29.2 ± 1.9**
Constraint-feasible oracle	24.1 ± 0.5	38.0 ± 0.2	16 ± 0.9	41.2 ± 1.6
Realme 11 Pro+ (Dimensity 7050)				
Fixed-Small	26.2 ± 0.7	38.4 ± 0.3	0	0.0
Fixed-Medium	71.5 ± 1.6	42.8 ± 0.4	0	0.0
Fixed-Large	162.0 ± 3.8	45.9 ± 0.6	0	0.0
Complexity-Only	49.8 ± 1.2	42.4 ± 0.4	44 ± 2.1	158.4 ± 6.8
Thermal-Aware	34.8 ± 0.9	39.5 ± 0.3	29 ± 1.8	86.5 ± 4.1
LCI v1 (reactive)	44.6 ± 1.1	41.2 ± 0.4	35 ± 2.0	122.4 ± 5.4
Haya (proposed)	**39.5 ± 0.9**	**39.8 ± 0.3**	**13 ± 1.2**	**44.8 ± 2.6**
Constraint-feasible oracle	33.5 ± 0.8	39.2 ± 0.2	18 ± 1.1	58.4 ± 2.4

Values after ± are the reported standard errors. The oracle uses offline sequence knowledge and is not deployable.

B. BREAK-EVEN AND CONTEXT-REBUILD COST

With $\lambda = 0.025$, the mix-averaged estimates of break-even turn counts are 1.1 for Gemma-2-2B, 1.2 for Qwen2.5-1.5B, 1.5 for Qwen2.5-3B, 3.0 for Phi-3-mini, and 3.3 for TinyLlama-1.1B (Fig. 4). For Phi-3-mini, the estimates vary from 1.3 turns for $\lambda = 0.020$ to 18.7 turns for $\lambda = 0.030$, and there is no break-even point for $\lambda \geq 0.0312$.

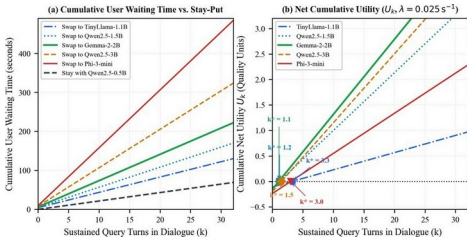


Fig. 4. Cumulative waiting time and utility after switching from Qwen2.5-0.5B; $\lambda = 0.025$ and break-even turns are marked.

Separation of flash load, initialization, and history re-prefill in Fig. 5 shows a swap time of 4.27 s, while 256 and 1,024 history tokens increase the time to 10.48 s and 29.12 s, correspondingly. At the observed 41.2-token/s rate of re-prefill, the cost of history in the long-dialogue case is 85%. Load-only overhead thus understates the cost of multi-turn switching.

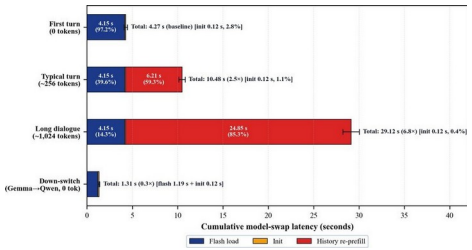


Fig. 5. Measured model-swap latency decomposed into flash load, initialization, and history re-prefill for different context lengths.

C. THERMAL RESPONSE

During ten minutes of continuous Qwen2.5-0.5B inference, chassis temperature readings were stabilized at 41 °C for GT 6T and 43 °C for Realme 11 Pro+ (Fig. 6). Continuous decoding throughput was 23% and 29% lower compared to burst rate, respectively. The thermal response delay for the external thermistor with respect to junction temperature changes was 3-5 seconds. Temperature increase during two minutes of heating for calibration was 7.0 °C and 8.2 °C, respectively (Table I). Such results indicate the need for action before thermal load resulting in throughput reduction; no measurement of CPU frequency changes was conducted directly.

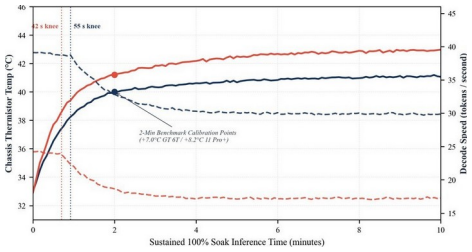


Fig. 6. Chassis temperature and decode throughput during continuous Qwen2.5-0.5B inference on the two phones.

D. PREEMPTIVE SWITCHING AND MEMORY RESIDENCY

Switching from Gemma-2-2B to Qwen2.5-0.5B was observed at around 45 s and 38 °C in the case of preemptive thermal test. As the time reaches around 72 s, reactive policy crossed above the threshold of 40 °C by reaching approximately 41.7 °C, while the preemptive policy maintained its temperature around 38.4–38.5 °C. In the separate test with an application budget of 2.8 GB, Qwen2.5-0.5B+Gemma-2-2B showed the value of 1,848 MB and Qwen2.5-0.5B+Phi-3-mini showed the value of 2,984 MB. The residency measurements of each of the three models was around 4,382 MB and exceeded the budget.

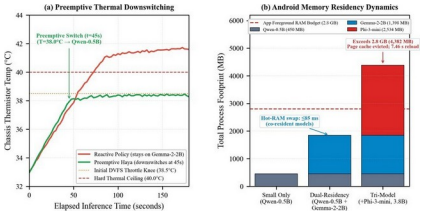


Fig. 7. Preemptive thermal downswitching and measured multi-model memory residency relative to the 2.8-GB app budget.

E. ABLATION STUDY

In Fig. 8, four quality components are extracted from the GT 6T. The complete Haya recording reports quality 0.746, 11 swaps, 3,253-ms average latency, and 38.5 °C peak temperature. Without the swap-cost factor, the quality is approximately 0.748, while swaps reach 38, latency 3,820 ms, and peak temperature 39.4 °C. Without thermal awareness, quality becomes approximately 0.742, while swaps, latency, and temperature become 13, 3,640 ms, and 42.4 °C, respectively. Without query fit, the quality is about 0.652, while swaps, latency, and temperature become 10, 3,440 ms, and 38.8 °C. The removal of hysteresis results in approximately 0.734 quality, 25 swaps, 3,580-ms latency, and 39.2 °C. Therefore, the query fit mainly ensures response quality, the swap cost reduces the overhead of migration, the thermal-aware aspect controls the heat, and finally hysteresis stops oscillation.

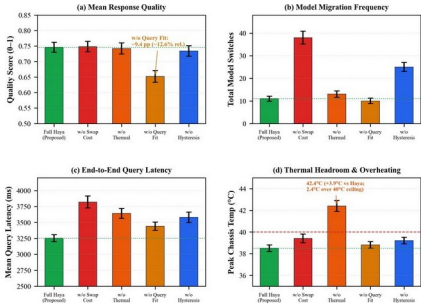


Fig. 8. Component ablation on the GT 6T: response quality, switches, mean latency, and peak chassis temperature.

F. SWAP-COST SENSITIVITY

For example, consider α multiplied by the exchange cost factor with other factors held constant. At α = 0.5, the router has 26 moves, with average energy consumption of 32.6 J/query, while at α = 1 as the benchmark has 11 moves with 28.8 J/query. Lastly, at α = 4, it has one move, approaching the fixed quality point. Increasing the Penalty factor makes tradeoff of quality and switch rate, i.e., latency and energy, more difficult, which is not necessarily a good feature. For example, increasing α from 1 to 2 improves latency by 513 ms (15.8%) and energy consumption by 4.0 J/query (13.9%), but reduces quality by 0.091 (Fig. 9).

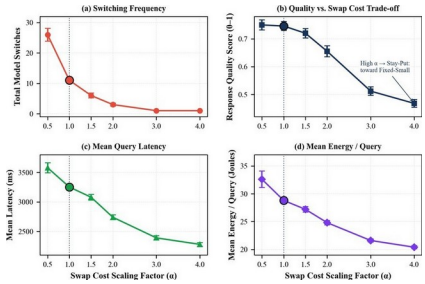


Fig. 9. Effect of swap-cost scale *α* on switches, quality, latency, and energy for the GT 6T.

VI. DISCUSSION AND FUTURE WORK

When swap cost was not considered, the number of transitions was more (increased from 11 to 38), and the average latency rose from 3,253 to 3,820 ms, although there was an increase in average quality by 0.002, while the cost associated with rebuild of 1,024 history tokens makes up 85% of the handoff cost. From Fig. 9, we can also observe one more danger – with the high swap cost, the router will make only one transition and reach the fixed-small quality level, thus, few transitions would be desirable in this situation. The importance of Haya lies in retaining reasonable transitions, and this is what break-even analysis detects.

For the experiments, two Realme phones were used, the CPU was chosen as the inference device, three tier routing and replay evaluation were performed. The quality of the results is measured by an LLM judge; The break-even utility is averaged across all prompts; the oracle can't be deployed. The reproducibility is limited because there are no metadata available including version of runtime used and the number of threads; source of prompts and the mechanism of operation of the judge; number of bootstraps; weight and normalizer of LCI; query-fit adjustment; hysteresis and the turn estimate. The comparisons of Haya against other relevant research are performed based on searches done when this paper was being written. Future work will entail recording the parameters mentioned above, weight optimization based on data, further testing on various smartphones, different workloads and users, history re-prefill versus KV-cache migration technique [7], and having a remote model that serves as a no-swap cost variant, which is an expansion to the offline policy, as the tier policy has already been addressed by HybridInfer [22].

VII. CONCLUSION

All the mentioned four components – per candidate LCI calculation, query matching, memory admission, and amortization of the cost of model loading and history

reconstruction – are incorporated into one swap policy by the Haya approach. In contrast with Complexity-Only approach that implies 200 prompt replay, the Haya approach achieves 29.6% lower latency and 17.2% lower power, while the number of hand-offs is reduced by 70%. Moreover, the average quality in case of Haya approach is 0.746, whereas, for other approaches, it is 0.724 and 0.718 respectively. The establishment of the history tokens for 1,024 tokens constitutes 85% of the total hand-off time. It is worth mentioning that the obtained results are relevant for two different devices with replay, and thus, can only prove the effectiveness of the proposed swap-aware approach.

ACKNOWLEDGEMENT

The wording of this paper was written by the authors by hand. For grammar purposes, it has been rephrased with the help of AI.

REFERENCES

[1] W. Yin, M. Xu, Y. Li, and X. Liu, “LLM as a system service on mobile devices,” arXiv:2403.11805, 2024.

[2] J. Xiao, Q. Huang, X. Chen, and C. Tian, “Understanding large language models in your pockets: Performance study on COTS mobile devices,” arXiv:2410.03613, 2024.

[3] E. Ehsani, L. Giamattei, I. Malavolta, and R. Pietrantuono, “Sustainability is not linear: Quantifying performance, energy, and privacy trade-offs in on-device intelligence,” arXiv:2603.26603, 2026.

[4] É. Guégain and T. Coignion, “The battery price of edge AI: A study of the environmental impact of LLM inference on mobile devices,” arXiv:2609.11940, 2026.

[5] R. Bao, N. Xue, Y. Sun, and Z. Chen, “Dynamic quality-latency aware routing for LLM inference in wireless edge-device networks,” arXiv:2508.11291, 2025.

[6] Y. Zhang *et al.*, “MTRouter: Cost-aware multi-turn LLM routing with history-model joint embeddings,” in *Proc. Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2026, arXiv:2604.23530.

[7] T. Heo *et al.*, “Cross-model KV cache transfer in LLM families: A closed-form linear mapping for prefill reuse,” arXiv:2608.03893, 2026.

[8] O. Sanseviero and I. Ballantyne, “Introducing Gemma 3n: The developer guide,” Google Developers Blog, Jun. 26, 2025. [Online]. Available: https://developers.googleblog.com/en/introducing-gemma-3n-developer-guide/

[9] Y. Yan, J. Shen, X. Luo, and Y. Zhou, “EdgeFlow: Fast cold starts for LLMs on mobile devices,” arXiv:2604.09083, 2026.

[10] W. Yin, R. Yi, D. Xu, G. Huang, M. Xu, and X. Liu, “Elastic on-device LLM service,” in *Proc. 31st Annu. Int. Conf. Mobile Comput. Netw. (MobiCom)*, Hong Kong, China, Nov. 2025, doi: 10.1145/3680207.3765259.

[11] Z. Liu *et al.*, “MobileLLM: Optimizing sub-billion parameter language models for on-device use cases,” arXiv:2402.14905, 2024.

[12] F. Tan *et al.*, “MobileQuant: Mobile-friendly quantization for on-device language models,” arXiv:2408.13933, 2024.

[13] K. Alizadeh *et al.*, “LLM in a flash: Efficient large language model inference with limited memory,” arXiv:2312.11514, 2023.

[14] Y. Song, Z. Mi, H. Xie, and H. Chen, “PowerInfer: Fast large language model serving with a consumer-grade GPU,” arXiv:2312.12456, 2023.

[15] Z. Xue, Y. Song, Z. Mi, X. Zheng, Y. Xia, and H. Chen, “PowerInfer-2: Fast large language model inference on a smartphone,” arXiv:2406.06282, 2024.

[16] H. Yu, M. Kim, J. Park, and K. Lee, “mzCache: On-device LLM memory management under multitasking,” *Proc. ACM Int. Conf. Mobile Comput. Netw. (MobiCom)*, 2026, arXiv:2609.01338, doi: 10.1145/3795866.3844495.

[17] I. Ong *et al.*, “RouteLLM: Learning to route LLMs with preference data,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2025, arXiv:2406.18665.

[18] L. Chen, M. Zaharia, and J. Zou, “FrugalGPT: How to use large language models while reducing cost and improving performance,” *Trans. Mach. Learn. Res.*, 2024, arXiv:2305.05176.

[19] D. Ding *et al.*, “Hybrid LLM: Cost-efficient and quality-aware query routing,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2024, arXiv:2404.14618.

[20] H. Jain, D. Pandey, and K. Vaidhyanathan, “CALM: A self-adaptive orchestration approach for QoS-aware routing in small language model based systems,” in *Proc. Int. Symp. Softw. Eng. Adapt. Self-Manag. Syst. (SEAMS)*, Rio de Janeiro, Brazil, Apr. 2026, arXiv:2602.03632.

[21] A. Matathammal, K. Gupta, L. Lavanya, A. V. Halgatti, P. Gupta, and K. Vaidhyanathan, “EdgeMLBalancer: A self-adaptive approach for dynamic model switching on resource-constrained edge devices,” arXiv:2502.06493, 2025.

[22] S. Koul, “HybridInfer: Thermal-aware reinforcement-learning tier routing for on-device, edge, and cloud LLM inference,” arXiv:2609.30270, 2026.

[23] A. Kabir, J. Xue, M. Zheng, and Q. Lou, “HW-Router: Hardware-aware routing for scalable multi-LLM serving,” arXiv:2608.14575, 2026.

[24] M. A. R. Siddiqui, D. Rojas, C. Yang, W. Cui, Y. Shi, and Y. Chen, “Measured joules, learned routes: Learning to route for energy-efficient LLM serving,” arXiv:2609.23085, 2026.

[25] B. Zou *et al.*, “EnerInfer: Energy-aware on-device LLM inference,” arXiv:2606.23001, 2026.

[26] E. Kim, Y. J. Jeon, and M. Han, “MemSpec: Memory-aware run-time for adaptive draft scheduling in speculative decoding on edge devices,” in *Proc. 27th ACM SIGPLAN/SIGBED Int. Conf. Lang., Compilers, Tools Embedded Syst. (LCTES)*, Jun. 2026, pp. 180–192, doi: 10.1145/3814943.3816174, arXiv:2608.10362.

[27] Google, “PowerManager: getThermalHeadroom(int forecastSeconds),” Android Developers. Accessed: Oct. 4, 2026. [Online]. Available: https://developer.android.com/reference/android/os/PowerManager

[28] realme, “realme GT 6T specifications.” Accessed: Oct. 6, 2026. [Online]. Available: https://www.realme.com/in/realme-gt-6t/specs

[29] realme, “realme 11 Pro+ specifications.” Accessed: Oct. 6, 2026. [Online]. Available: https://www.realme.com/sg/realme-11-pro-plus/specs

[30] Qwen *et al.*, “Qwen2.5 technical report,” arXiv:2412.15115, 2024.

[31] Gemma Team *et al.*, “Gemma 2: Improving open language models at a practical size,” arXiv:2408.00118, 2024.

[32] M. Abdin *et al.*, “Phi-3 technical report: A highly capable language model locally on your phone,” arXiv:2404.14219, 2024.

[33] ggml-org, “llama.cpp: LLM inference in C/C++,” GitHub repository. Accessed: Oct. 6, 2026. [Online]. Available: https://github.com/ggml-org/llama.cpp

[34] ggml-org, “GGUF file format specification,” GitHub repository. Accessed: Oct. 6, 2026. [Online]. Available: https://github.com/ggml-org/ggml/blob/master/docs/gguf.md

[35] L. Zheng *et al.*, “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, arXiv:2306.05685.

[36] J. Cohen, “A coefficient of agreement for nominal scales,” *Educ. Psychol. Meas.*, vol. 20, no. 1, pp. 37–46, 1960, doi: 10.1177/001316446002000104.